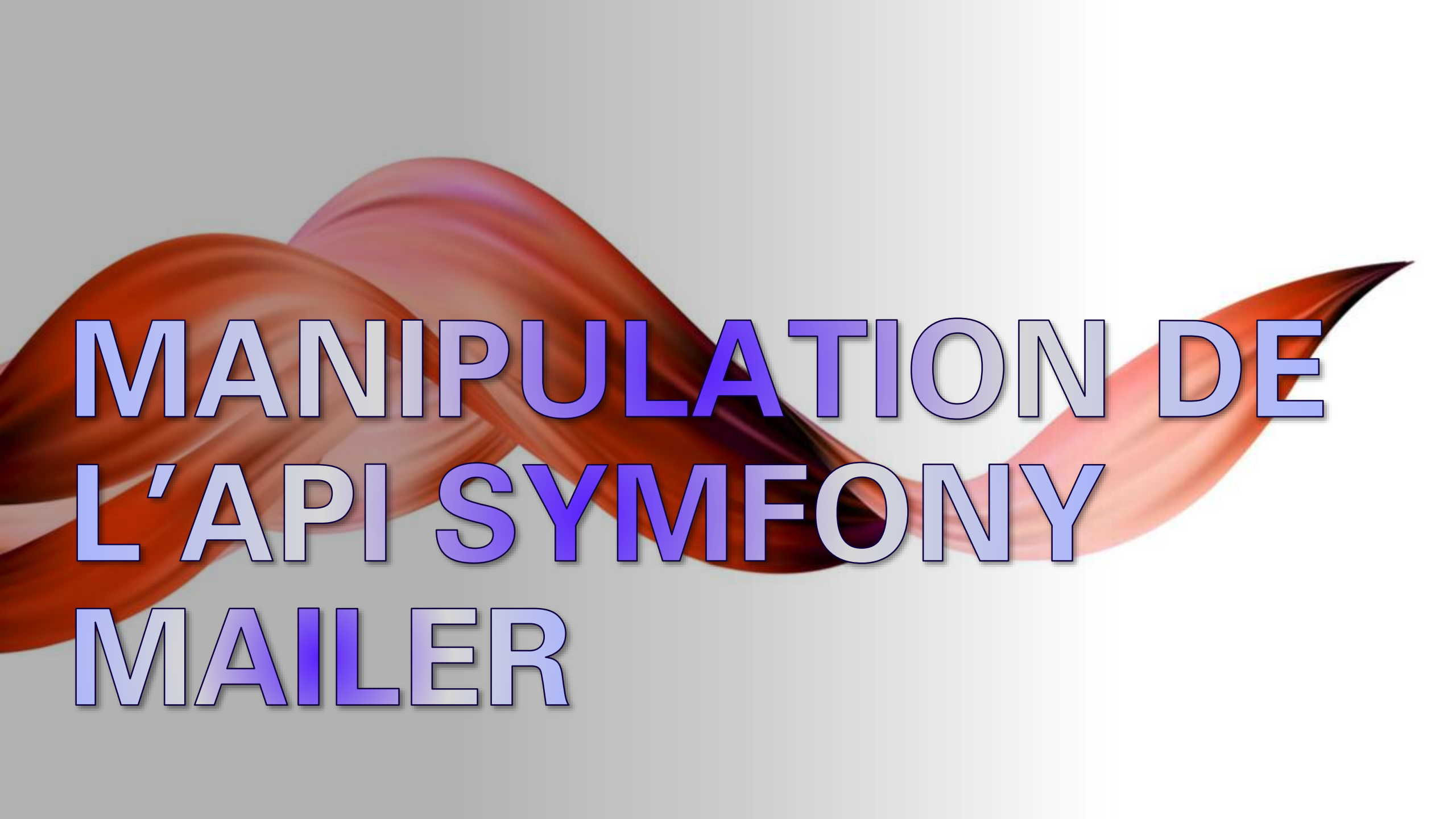




MANIPULATION DE L'API SYMFONY MAILER

Introduction

L'envoi d'e-mails ne doit pas être compliqué. Laravel fournit une API de messagerie simple et propre, alimentée par le composant populaire **Symfony Mailer**.

Laravel et Symfony Mailer fournissent des pilotes pour l'envoi d'e-mails via **SMTP, Mailgun, Postmark, Amazon SES** et **sendmail**, ce qui vous permet de commencer rapidement à envoyer des e-mails via un service local ou basé sur le cloud de votre choix.

Configuration

Les services de messagerie de Laravel peuvent être configurés via le fichier de configuration **config/mail.php** de votre application.

Dans votre fichier de configuration de messagerie, vous trouverez un tableau de configuration **mailers**. Ce tableau contient un exemple d'entrée de configuration pour chacun des principaux pilotes/transports de messagerie pris en charge par Laravel, tandis que la valeur de configuration par défaut détermine le mailer qui sera utilisé par défaut lorsque votre application devra envoyer un message électronique.

- +
-
-

GÉNÉRATION DE CLASSES MAILABLE

La classe mailable

Lorsque vous construisez des applications Laravel, chaque type d'e-mail envoyé par votre application est représenté par une classe "**mailable**". Ces classes sont stockées dans le répertoire **app/Mail**. Ne vous inquiétez pas si vous ne voyez pas ce répertoire dans votre application, car il sera généré pour vous lorsque vous créerez votre première classe mailable à l'aide de la commande **Artisan make:mail** :

```
php artisan make:mail OrderShipped
```


La classe mailable

Une fois que vous avez généré une classe mailable, ouvrez-la afin que nous puissions explorer son contenu. La configuration de la classe mailable s'effectue selon plusieurs méthodes, dont celles **envelope**, **content** et **attachements**.

La méthode **envelope** renvoie un objet **Illuminate\Mail\Mailables\Envelope** qui définit l'objet et, parfois, les destinataires du message.

La méthode **content** renvoie un objet **Illuminate\Mail\Mailables\Content** qui définit le modèle Blade qui sera utilisé pour générer le contenu du message.

Configuration de l'expéditeur

Tout d'abord, explorons la configuration de l'expéditeur de l'e-mail. Il existe deux façons de configurer l'expéditeur.

Premièrement, vous pouvez spécifier l'adresse "**from**" dans la méthode **envelope** de votre message :

```
public function envelope()  
{  
    return new Envelope(  
        from: new Address('jeffrey@example.com', 'Jeffrey Way'),  
        subject: 'Order Shipped',  
    );  
}
```

Configuration de la vue

Dans la méthode **content** d'une classe **mailable**, vous pouvez définir la vue ou le modèle à utiliser pour le rendu du contenu de l'e-mail. Étant donné que chaque courriel utilise généralement un modèle Blade pour afficher son contenu, vous bénéficiez de toute la puissance et de la commodité du moteur de modèles Blade lorsque vous créez le code HTML de votre courriel :

```
public function content()
{
    return new Content(
        view: 'emails.orders.shipped',
    );
}
```


Affichage des données

Vous pouvez mettre des données à la disposition de votre public de deux manières. Tout d'abord, toute propriété publique définie sur votre classe **mailable** sera automatiquement mise à disposition de la vue.

```
class OrderShipped extends Mailable
{
    public $order;

    public function __construct(Order $order)
    {
        $this->order = $order;
    }
    ....
}
```

Affichage des données

Une fois que les données ont été définies comme propriété publique, elles seront automatiquement disponibles dans votre vue, de sorte que vous pouvez y accéder comme vous le feriez pour toute autre donnée dans vos modèles Blade :

```
<div>  
    Price: {{ $order->price }}  
</div>
```

Affichage des données

Si vous souhaitez personnaliser le format des données de votre message avant qu'il ne soit envoyé au modèle, vous pouvez transmettre manuellement vos données à la vue via le paramètre **with** de la définition de contenu.

```
public function content()
{
    return new Content(
        view: 'emails.orders.shipped',
        with: [
            'orderName' => $this->order->name,
            'orderPrice' => $this->order->price,
        ],
    );
}
```

Pièces jointes

Pour ajouter des pièces jointes à un message électronique, vous devez les ajouter au tableau renvoyé par la méthode **Attachments** du message. Tout d'abord, vous pouvez ajouter une pièce jointe en fournissant un chemin de fichier à la méthode **fromPath** fournie par la classe **Attachment** :

```
public function attachments()  
{  
    return [  
        Attachment::fromPath('/path/to/file'),  
    ];  
}
```

Attacher des fichiers à partir d'un disque

Si vous avez stocké un fichier sur l'un des disques de votre système de fichiers, vous pouvez le joindre à l'e-mail en utilisant la méthode **fromStorage** :

```
public function attachments()  
{  
    return [  
        Attachment::fromStorage('/path/to/file'),  
        ->as('name.pdf')  
    ];  
}
```



+

•

○

ENVOI DE COURRIER

Envoi de courrier

Pour envoyer un message, utilisez la méthode **to** de la façade **Mail**. La méthode **to** accepte une adresse électronique, une instance d'utilisateur ou une collection d'utilisateurs. Si vous transmettez un objet ou une collection d'objets, l'expéditeur utilisera automatiquement leurs propriétés email et nom pour déterminer les destinataires du message. Une fois que vous avez spécifié vos destinataires, vous pouvez passer une instance de votre classe **mailable** à la méthode d'envoi :

```
Mail::to($request->user())->send(new OrderShipped($order));  
// or  
Mail::to('testmail@mail.com')->send(new OrderShipped($order));
```

Envoi de courrier

Vous n'êtes pas limité à la spécification des destinataires "**to**" lorsque vous envoyez un message. Vous êtes libre de définir les destinataires "**to**", "**cc**" et "**bcc**" en enchaînant leurs méthodes respectives :

```
Mail::to($request->user())  
    ->cc($moreUsers)  
    ->bcc($evenMoreUsers)  
    ->send(new OrderShipped($order));
```

Envoi d'Email via un expéditeur spécifique

Par défaut, Laravel enverra le courrier électronique en utilisant le mailer configuré comme mailer par défaut dans le fichier de configuration du courrier de votre application. Toutefois, vous pouvez utiliser la méthode **mailer** pour envoyer un message en utilisant une configuration mailer spécifique :

```
Mail::mailer('postmark')  
    ->to($request->user())  
    ->send(new OrderShipped($order));
```